

# Differential Dynamic Programming

## Details and Constraints

### Optimization and Learning for Robot Control

RCI Lab

Kyung Hee University

February 21, 2026

Nonlinear trajectory optimization:

$$\min_{\{x_k, u_k\}} \sum_{k=0}^{N-1} \ell(x_k, u_k) + \ell_f(x_N)$$

subject to:

$$x_{k+1} = f(x_k, u_k)$$

Backward pass approximates:

$$Q_k(x, u) = \ell(x, u) + V_{k+1}(f(x, u))$$

## Second Order Expansion of $Q$

Expand around nominal  $(\bar{x}, \bar{u})$ :

$$Q(x, u) \approx Q + Q_x \delta x + Q_u \delta u + \frac{1}{2} \begin{bmatrix} \delta x \\ \delta u \end{bmatrix}^T \begin{bmatrix} Q_{xx} & Q_{xu} \\ Q_{ux} & Q_{uu} \end{bmatrix} \begin{bmatrix} \delta x \\ \delta u \end{bmatrix}$$

Optimality:

$$Q_u + Q_{ux} \delta x + Q_{uu} \delta u = 0$$

$$\delta u = k + K \delta x$$

$$k = -Q_{uu}^{-1} Q_u$$

$$K = -Q_{uu}^{-1} Q_{ux}$$

# Value Function Update (Detailed)

Plug optimal  $\delta u$  back:

$$V(x + \delta x) = \min_{\delta u} Q(x + \delta x, u + \delta u)$$

First derivative:

$$V_x = Q_x - Q_{xu} Q_{uu}^{-1} Q_u$$

Second derivative:

$$V_{xx} = Q_{xx} - Q_{xu} Q_{uu}^{-1} Q_{ux}$$

This is a Schur complement.

DDP  $\simeq$  Newton in trajectory space.

# Regularization (Stability)

Problem:

$$Q_{uu} \neq 0$$

Newton step unstable.

Add Levenberg-Marquardt regularization:

$$Q_{uu} \leftarrow Q_{uu} + \lambda I$$

Interpretation:

- Large  $\lambda \rightarrow$  gradient descent
- Small  $\lambda \rightarrow$  Newton

Adjust  $\lambda$  adaptively.

# Forward Pass with Line Search

New control:

$$u_k^{new} = u_k + \alpha k_k + K_k(x_k^{new} - x_k)$$

$$\alpha \in (0, 1]$$

Check cost decrease:

$$J(\alpha) < J(0)$$

Predicted decrease:

$$\Delta J = \alpha Q_u^T k + \frac{1}{2} \alpha^2 k^T Q_{uu} k$$

# Control Constraints

Original DDP assumes unconstrained  $u$ .

But in robotics:

$$u_{min} \leq u \leq u_{max}$$

Solve constrained quadratic subproblem:

$$\min_{\delta u} Q_u^T \delta u + \frac{1}{2} \delta u^T Q_{uu} \delta u$$

subject to:

$$u + \delta u \in [u_{min}, u_{max}]$$

# Box-Constrained DDP

Active-set strategy:

1. Detect saturated controls 2. Split into free and clamped components 3.

Solve reduced system

Partition:

$$\delta u = \begin{bmatrix} \delta u_f \\ \delta u_c \end{bmatrix}$$

Solve only for free part:

$$\delta u_f = -Q_{uu,ff}^{-1}(Q_{u,f} + Q_{ux,f}\delta x)$$

Full DDP includes:

$$\sum_i V'_{x_i} f_{uu}^{(i)}$$

Gauss-Newton approximation drops second-order dynamics:

$$Q_{uu} = \ell_{uu} + f_u^T V'_{xx} f_u$$

Advantages:

- Positive semi-definite
- More stable
- Cheaper

Widely used in robotics.

Nonlinear programming form:

$$\min_z F(z)$$

$$\text{s.t. } c(z) = 0$$

SQP step:

$$\min_{\Delta z} \nabla F^T \Delta z + \frac{1}{2} \Delta z^T H \Delta z$$

$$\text{s.t. } J_c \Delta z + c = 0$$

DDP = structured SQP exploiting dynamics.

# DDP vs iLQR

| DDP                      | iLQR         |
|--------------------------|--------------|
| Full 2nd order dynamics  | Gauss-Newton |
| Faster local convergence | More stable  |
| More expensive           | Cheaper      |

Both are:

$$O(Nn^3)$$

Very efficient for robotic systems.

- Warm-start from previous MPC solve
- Regularization critical for humanoids
- Contact dynamics increases non-convexity
- Often combined with shooting methods

Used in:

- Legged robots
- Manipulators
- Humanoid control